

Hallucination Detection in Tool Calling

Task description

In this task, you will build a dataset for detecting hallucinations in dialogs with tool calling, evaluate the performance of existing methods, and improve their results. To address this task, you should:

1. Inject hallucinations into the existing dialogue dataset containing tool calling
2. Evaluate existing baseline methods
3. Improve the baseline approaches

1. Inject hallucinations into the existing dialogue dataset containing tool calling

As the base dataset, we propose to use the ToolACE dataset

(<https://arxiv.org/abs/2409.00920v2>). This dataset consists of dialogues that include the following components: a user query, a list of available tools, a tool calling, the tool's output, and the model's final response. You should automatically corrupt the dataset by injecting the following types of hallucinations::

- 1) Hallucination between the model's response and the tool's output:
User: Help me check the weather in Beijing.
Tool answer: Weather_API:{location: "Beijing", weather: "sunny"}
Answer: *The weather in Beijing is rainy. [hallucination]*
- 2) Overgeneration, when the response contains information that is not present in the tool's output:
User: Help me check the weather in Beijing.
Tool answer: Weather_API:{location: "Beijing", weather: "sunny"}
Answer: *The weather in Beijing is sunny, and the weather has been pretty good past few months [overgeneration]*
- 3) Missing tool, when the response recommends actions that require calling a tool that is not available:
User: Help me check the weather in Beijing.
Tools: Weather_API
Tool answer: Weather_API:{location: "Beijing", weather: "sunny"}
Answer: *The weather in Beijing is sunny, would you like me to book a ticket? [missing tool]*

Thus, at the end of this step you will have three datasets with different types of span-based hallucinations. The annotation of hallucination should be span-based and stored in RAGTruth format (<https://huggingface.co/datasets/wandb/RAGTruth-processed>), where

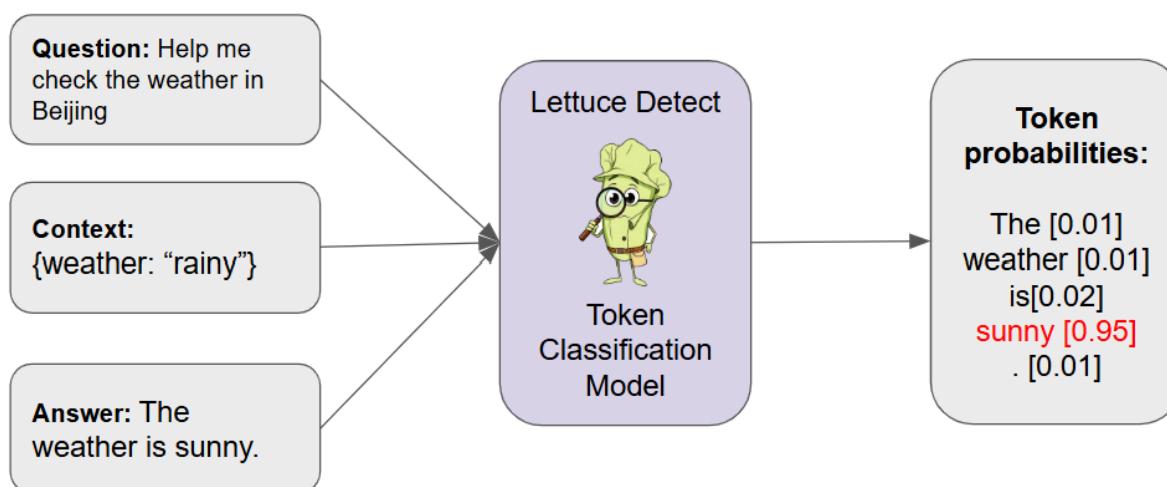
- Query = user query
- Context = tool response
- Output = model final answer
- Hallucination_labels - span labeled hallucinations, that you added to the initial dataset

2. Evaluate existing baseline methods

On the resulting datasets, evaluate the existing baseline methods for detecting hallucinations in language model responses:

- 1) LettuceDetect (<https://arxiv.org/pdf/2502.17125>) a BERT-based method.
- 2) LookBackLens (<https://arxiv.org/abs/2407.07071>) an LLM-based method that measures how much attention is paid to context during response generation and uses a classifier built on these signals.

Use the user query as the question, the tool output as the context, and the model's final response as the answer. After this step, you will obtain evaluation results for each dataset and method. An example of the input data and the expected answer is shown below:



3. Improve the baseline approaches

In this step, develop methods to improve the baseline approaches; you may use additional data, refine the methods, or apply other enhancements. By the end of this step, you will have a set of experiments evaluating the new approaches across the three datasets, each corresponding to a different hallucination type.

Results

Dataset and methodology: a dataset for span-level hallucination detection was prepared; the dataset creation procedure based on the original ToolACE is described, including data selection, annotation, and preprocessing steps. The dataset is published on Hugging Face.

Model: a span classification model was trained to identify and classify text spans and detect hallucinations. Training and a basic evaluation were conducted. The model is available on Hugging Face.

Experiments: a set of experiments was conducted to evaluate the proposed model; improvements over baseline models are demonstrated, and a results table along with code for reproducing the experiments is provided.